

Understand Java programming Basics

Introduction to Java Introduction to Java

What Is Java?

> Java is an object-oriented programming language developed by Sun Microsystems, a company best known for its high-end Unix workstations. Modeled after C++, the Java language was designed to be small, simple, and portable across platforms and operating systems, both at the source and at the binary level.

Brief History of Java:

> James Gosling initiated the Java language project in June 1991 for use in one of his many set-top box projects. The language, initially called Oak after an oak tree that stood outside Gosling's office, also went by the name Green and ended up later being renamed as Java, from a list of random words.

Sun released the first public implementation as Java 1.0 in 1995. It promised Write Once, Run Anywhere (WORA), providing no-cost run-times on popular platforms. On 13 November 2006, Sun released much of Java as free and open source software under the terms of the GNU General Public License (GPL).

On 8 May 2007, Sun finished the process, making all of Java's core code free and open-source, aside from a small portion of code to which Sun did not hold the copyright.

Tools you will need:

You will need a Pentium 200-MHz computer with a minimum of 64 MB of RAM (128 MB of RAM recommended).

You also will need the following softwares:

|| Linux 7.1 or Windows 95/98/2000/XP operating system.

|| Java JDK 5

|| Microsoft Notepad or any other text editor

This tutorial will provide the necessary skills to create GUI, networking, and Web applications using Java.

Popular Java Editors:

To write your Java programs, you will need a text editor. There are even more

sophisticated IDEs available in the market. But for now, you can consider one of the following:

- || Notepad: On Windows machine, you can use any simple text editor like Notepad (Recommended for this tutorial), TextPad.
- || Netbeans: Is a Java IDE that is open-source and free which can be downloaded from <http://www.netbeans.org/index.html>.
- || Eclipse: Is also a Java IDE developed by the eclipse open-source community and can be downloaded from <http://www.eclipse.org/>.

Java component and purpose

✓ *Basic Components of Java Programs*

Java programs consist of several basic components that work together to create a functional application. Here are the key components:

1. Classes

- *Definition*: A class is a blueprint or template that defines the properties and behavior of an object.
- *Purpose*: Classes define the structure and behavior of objects, including variables, methods, and constructors.

2. Objects

- *Definition*: An object is an instance of a class, representing a real-world entity or concept.
- *Purpose*: Objects have their own set of attributes (data) and methods (functions) that define their behavior.

3. Methods

- *Definition*: A method is a block of code that performs a specific task or operation.
- *Purpose*: Methods define the behavior of objects and classes, allowing them to interact with each other and perform actions.

4. Variables

- *Definition*: Variables are used to store and manipulate data in a Java program.
- *Types*: Java has various types of variables, including primitive types (e.g., int, boolean) and reference types (e.g., objects, arrays).

5. Constructors

- ***Definition***: A constructor is a special method used to initialize objects when they are created.
- ***Purpose***: Constructors set the initial state of an object by assigning values to its attributes.

6. Packages

- ***Definition***: A package is a collection of related classes and interfaces.
- ***Purpose***: Packages help organize and structure Java code, making it easier to manage and reuse.

7. Interfaces

- ***Definition***: An interface is an abstract class that defines a contract or set of methods that must be implemented.
- ***Purpose***: Interfaces define a common set of methods that can be implemented by multiple classes.

8. Main Method

- ***Definition***: The main method is the entry point of a Java program, where execution begins.
- ***Purpose***: The main method is responsible for launching the program and executing its logic.

These basic components work together to create a functional Java program that can perform a wide range of tasks and operations.

✓ ***Java Constructs***

Java constructs refer to the basic building blocks of the Java programming language. Here are some key Java constructs:

1. Variables and Data Types

- ***Variables***: Store and manipulate data in a Java program.
- ***Data Types***: Define the type of data a variable can hold, such as int, double, boolean, etc.

2. Operators

- **Arithmetic Operators**: Perform mathematical operations, such as +, -, *, /, etc.
- **Comparison Operators**: Compare values, such as ==, !=, >, <, etc.
- **Logical Operators**: Combine conditions, such as &&, ||, !, etc.

3. Control Structures

- **Conditional Statements**: if-else statements that execute code based on conditions.
- **Loops**: for, while, and do-while loops that execute code repeatedly.
- **Switch Statements**: Execute code based on the value of a variable.

4. Methods

- **Definition**: A block of code that performs a specific task or operation.
- **Purpose**: Methods define the behavior of objects and classes.

5. Classes and Objects

- **Classes**: Define the structure and behavior of objects.
- **Objects**: Instances of classes, representing real-world entities or concepts.

6. Arrays and Collections

- **Arrays**: Store multiple values of the same type in a single variable.
- **Collections**: Store and manipulate groups of objects, such as lists, sets, and maps.

Object Declaration vs Object Creation

Here's the difference between object declaration and object creation:

Object Declaration

- **Definition**: Declaring an object means specifying the type of object and its name.
- **Example**: `String str;`
- **Purpose**: Object declaration informs the compiler about the object's type and name.

Object Creation

- **Definition**: Creating an object means allocating memory for the object and

initializing it.

- *Example*: `String str = new String("Hello");`
- *Purpose*: Object creation brings the object into existence and allocates memory for it.

Key Differences

1. *Memory Allocation*: Object declaration does not allocate memory, while object creation does.

}
2. *Initialization*: Object declaration does not initialize the object, while object creation does.
3. *Usage*: An object can be declared without being created, but it cannot be used until it is created.

In summary, object declaration is about specifying the object's type and name, while object creation is about allocating memory and initializing the object.

Instantiation in Java

Instantiation in Java refers to the process of creating an instance of a class, which is an object that has its own set of attributes (data) and methods (functions). When you instantiate a class, you create a new object that has its own copy of the class's variables and methods.

Describe Java insatiable classes and objects

Java Classes and Objects

Classes

- *Definition*: A class is a blueprint or template that defines the properties and behavior of an object.
- *Purpose*: Classes define the structure and behavior of objects, including variables, methods, and constructors.
- *Components*:
 1. *Fields* (variables): Define the properties of an object.
 2. *Methods*: Define the behavior of an object.
 3. *Constructors*: Initialize objects when they are created.

Objects

- ***Definition*:** An object is an instance of a class, representing a real-world entity or concept.
- ***Purpose*:** Objects have their own set of attributes (data) and methods (functions) that define their behavior.
- ***Characteristics*:**

1. ***State*:** An object's state is defined by its attributes (data).
2. ***Behavior*:** An object's behavior is defined by its methods (functions).

Relationship between Classes and Objects

- ***Instantiation*:** An object is created from a class using the `new` keyword.
- ***Inheritance*:** Classes can inherit properties and behavior from parent classes.

Benefits

1. ***Modularity*:** Classes and objects promote modular code, making it easier to maintain and reuse.
2. ***Reusability*:** Classes and objects can be reused across applications, reducing development time.
3. ***Flexibility*:** Classes and objects support complex and dynamic systems, making them suitable for a wide range of applications.

Example

```
public class Person {  
    private String name;  
    private int age;  
  
    public Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
}
```

```
// Instantiating the Person class  
Person person = new Person("John", 30);
```

Basic Syntax:

About Java programs, it is very important to keep in mind the following points.

- || Case Sensitivity - Java is case sensitive, which means identifier Welcome and welcome would have different meaning in Java.

- || Class Names - For all class names, the first letter should be in Upper Case.

If several words are used to form a name of the class, each inner word's first letter should be in Upper Case.

Example class MyFirstJavaClass

- || Method Names - All method names should start with a Lower Case letter.

If several words are used to form the name of the method, then each inner word's first letter should be in Upper Case.

Example public void myMethodName()

- || Program File Name - Name of the program file should exactly match the class name.

When saving the file, you should save it using the class name (Remember Java is case sensitive) and append '.java' to the end of the name (if the file name and the class name do not match your program will not compile).

Example : Assume 'MyFirstJavaProgram' is the class name, then the file should be saved as 'MyFirstJavaProgram.java'

- || public static void main(String args[]) - Java program processing starts from the main() method, which is a mandatory part of every Java program.

Java Identifiers:

All Java components require names. Names used for classes, variables and methods are called identifiers. In Java, there are several points to remember about identifiers. They are as follows:

- || All identifiers should begin with a letter (A to Z or a to z), currency character (\$) or an underscore (_).

- || After the first character, identifiers can have any combination of characters.

- || A keyword cannot be used as an identifier.
- || Most importantly identifiers are case sensitive.
- || Examples of legal identifiers: age, \$salary, _value, __l_value
- || Examples of illegal identifiers: 123abc, -salary

Java Modifiers:

Like other languages, it is possible to modify classes, methods, etc., by using modifiers. There are two categories of modifiers:

In Java, modifiers are keywords that alter or modify the behavior or accessibility of a variable, method, or class. They provide additional information about the declaration and help the compiler understand the intended usage.

Here are the types of modifiers in Java:

1. *Access Modifiers*: control access to classes, methods, and variables

- public
- private
- protected
- default (no modifier)

2. *Non-Access Modifiers*: provide additional information about the declaration

- static
- final
- abstract
- synchronized
- volatile
- transient
- strictfp

3. *Class Modifiers*: apply to classes and interfaces

- abstract
- final
- public
- private
- protected

4. *Method Modifiers*: apply to methods

- abstract
- final
- static
- synchronized
- native
- strictfp

5. *Variable Modifiers*: apply to variables

- final
- static
- transient
- volatile

Modifiers can be combined in various ways to achieve the desired behavior. For example:

```
...  
public static final int MAX_VALUE = 100;  
...
```

Here, 'public', 'static', and 'final' are modifiers that combine to define the accessibility and behavior of the 'MAX_VALUE' variable.

Understanding modifiers is essential in Java programming, as they help define the structure and behavior of your code.

Java Variables:

In Java, a variable is a named storage location that holds a value. Variables have a specific data type, and they are used to store and manipulate data in a program.

Here are the key aspects of variables in Java:

1. ***Declaration***: Variables must be declared before they can be used. Declaration includes specifying the data type and name of the variable.
2. ***Data Type***: The data type determines the type of value a variable can hold, such as integer, floating-point number, or string.
3. ***Name***: The name of the variable, also known as the identifier, must follow Java's naming conventions.
4. ***Assignment***: A value is assigned to a variable using the assignment operator (`=`).
5. ***Scope***: Variables have a scope, which determines their visibility and accessibility within the program.
6. ***Lifetime***: Variables have a lifetime, which determines how long they exist in memory.

Types of Variables in Java:

1. ***Local Variables***: Declared within a method or block, and are destroyed when the method or block is exited.
2. ***Instance Variables***: Declared within a class, and are associated with an object.
3. ***Static Variables***: Declared within a class, and are shared by all objects of the class.
4. ***Parameter Variables***: Declared in a method signature, and are used to pass values to a method.
5. ***Array Variables***: Used to store collections of values of the same data type.

Java Variable Example:

```
int myVariable; // Declare a variable

myVariable = 10; // Assign a value

System.out.println(myVariable); // Print the value
```

In this example, 'myVariable' is a variable of type 'int', assigned the value '10', and then printed to the console.

Java Arrays

Arrays are objects that store multiple variables of the same type. However, an array itself is an object on the heap.

How to Guide students to identify different components of java and compilation of a java

Here's a step-by-step guide to help students identify different components of Java and understand the compilation process:

Identifying Java Components:

1. ***Classes:*** Look for keywords like "public class" or "class" followed by a name (e.g., "public class HelloWorld").
2. ***Methods:*** Identify keywords like "public static void" or "private int" followed by a method name (e.g., "public static void main(String[] args)").
3. ***Variables:*** Recognize data types like "int", "String", or "boolean" followed by a variable name (e.g., "int x = 5").
4. ***Operators:*** Spot symbols like "+", "-", "*", "/", "=", etc. used for arithmetic, comparison, or assignment operations.
5. ***Control Flow Statements:*** Identify keywords like "if", "else", "for", "while", "switch", etc. that control the program's flow.

Exercise:

1. Write a Java code in a text editor or IDE (Integrated Development Environment) like Eclipse or Netbeans.
2. ***Save as java File:*** Save the code with a ".java" extension (e.g., "WelcomeToCom123.java").
3. ***Compile using javac:*** Use the "javac" command to compile the "java" file into a ".class" file (e.g., "javac WelcomeToCom123.java").
4. ***Generated .class File:*** A ".class" file is generated, containing bytecode that

can be run on the Java Virtual Machine (JVM).

5. *Run using java:* Use the "java" command to run the compiled ".class" file (e.g., "java WelcomToCom123").

Tips:

- Use a Java IDE to write, compile, and run code.
- Understand the difference between ".java" (source code) and ".class" (compiled bytecode) files.
- Familiarize yourself with Java syntax and basic constructs.
- Practice writing and compiling simple Java programs to reinforce understanding.

By following this guide, students should be able to identify different components of Java and understand the compilation process.

Java Keywords:

The following list shows the reserved words in Java. These reserved words may not be used as constant or variable or any other identifier names.

abstract	assert	boolean	break
byte	case	catch	char
class	const	continue	default
do	double	else	enum
extends	final	finally	float
for	goto	if	implements
import	instanceof	int	interface
long	native	new	package
private	protected	public	return
short	static	strictfp	super
switch	synchronized	this	throw
throws	transient	try	void
volatile	while		

Comments in Java

Java supports single-line and multi-line comments very similar to c and c++. All characters available inside any comment are ignored by Java compiler.

```
public class MyFirstJavaProgram{
```

```

/* This is my first java program.
 * This will print 'Welcome to come 123' as the output
 * This is an example of multi-line comments.
 */

public static void main(String[] args) {
    // This is an example of single line comment
    /* This is also an example of single line comment. */
    System.out.println("Welcome to Com 123");
}
}

```

Java Object & Classes

Java is an Object-Oriented Language. As a language that has the Object Oriented feature, Java supports the

following fundamental concepts:

- || Polymorphism
- || Inheritance
- || Encapsulation
- || Abstraction
- || Classes
- || Objects
- || Instance
- || Method
- || Message Parsing

The concepts Classes and Objects.

|| Object - Objects have states and behaviors. Example: A dog has states-color, name, breed as well as behaviors -wagging, barking, eating. An object is an instance of a class.

Let us now look deep into what are objects. If we consider the real-world we can

find many objects around us, Cars, Dogs, Humans, etc. All these objects have a state and behavior.

If we consider a dog, then its state is - name, breed, color, and the behavior is - barking, wagging, running

Here's an example of an object in Java:

Let's say we have a class called `Car` with the following attributes and methods:

- Attributes: 

- `color` (String)

- `speed` (int)

- Methods:

- `accelerate()` (increases speed by 10)

- `getSpeed()` (returns the current speed)

Now, let's create an object from this class:

```
Car myCar = new Car("Red");`
```

In this example:

- `myCar` is the object

- `Car` is the class (the blueprint)

- `"Red"` is the value passed to the constructor to initialize the `color` attribute

This object `myCar` has its own state (attributes) and behavior (methods):

- State:

- `color` = "Red"

- `speed` = 0 (default value)

- Behavior:

- `accelerate()` method can be called to increase the speed

- `getSpeed()` method can be called to retrieve the current speed

We can manipulate the object like this:

```
`myCar.accelerate();` // increases speed to 10
```

```
`int currentSpeed = myCar.getSpeed();` // returns 10
```

In this way, objects have their own unique state and behavior, which is defined by the class from which they're created.

|| Class - In Java, a class is a blueprint or a template that defines the structure and behavior of an object. It's a fundamental concept in object-oriented programming (OOP).

A class represents a real-world entity or concept, such as a car, a student, or an employee. It defines:

1. ***State*** (attributes): Data members or fields that describe the characteristics of the entity.
2. ***Behavior*** (methods): Actions that can be performed by the entity.
3. ***Identity*** (constructors): Special methods that initialize objects when they're created.

Classes in Java typically include:

- ***Fields*** (data members): Variables that store information about the object.
- ***Methods*** (functions): Blocks of code that perform actions on the object.
- ***Constructors***: Special methods that initialize objects when they're created.
- ***Modifiers***: Keywords that define access levels, such as public, private, or protected.

Classes are essentially templates for creating objects. When you create an object from a class, you're said to be instantiating the class.

Here's a simple example:

...

```
public class Car {  
    // Fields  
    private String color;  
    private int speed;  
    // Constructor  
    public Car(String color) {  
        this.color = color;  
    }  
}
```

```

    }

    // Methods

    public void accelerate() {

        speed += 10; }

    public int getSpeed() {

        return speed;

    }

}

```

This Car class defines a template for creating car objects, with fields for color and speed, a constructor to initialize the color, and methods to accelerate and get the speed.

Constructors:

When discussing about classes, one of the most important subtopic would be constructors. Every class has a constructor. If we do not explicitly write a constructor for a class the Java compiler builds a default constructor for that class.

Each time a new object is created, at least one constructor will be invoked. The main rule of constructors is that they should have the same name as the class. A class can have more than one constructor.

Example of a constructor is given below:

```

public class Puppy{

    public Puppy(){

    }

    public Puppy(String name){

    // This constructor has one parameter, name.

    }

}

```

Object declaration and object creation are two separate concepts in Java:

Object Declaration:

- Declaring a reference variable of a class type

- No memory is allocated for the object
- Example: ``Car myCar;`` (only declares a variable, no object exists yet)

Object Creation:

- Creating an instance of a class using the ``new`` keyword
- Memory is allocated for the object
- Example: ``Car myCar = new Car();`` (creates a new Car object and assigns it to the variable)

Key differences:

- Declaration only declares a variable, while creation instantiates an object.
- Declaration doesn't allocate memory, while creation does.
- Declaration can be done without the ``new`` keyword, while creation requires it.

Here's a step-by-step breakdown:

1. Object Declaration:

- ``Car myCar;`` // declares a variable, no object exists yet

2. Object Creation:

- ``myCar = new Car();`` // creates a new Car object and assigns it to the variable

By combining both steps, you can declare and create an object in one line:

```
* `Car myCar = new Car();` // declares and creates a new Car object
```

Based on the data type of a variable, the operating system allocates memory and decides what can be stored in the reserved memory. Therefore, by assigning different data types to variables, you can store integers, decimals, or characters in these variables.

Java Basic Data Types

There are two data types available in Java:

|| Primitive Data Types

|| Reference/Object Data Types

Primitive Data Types:

There are eight primitive data types supported by Java. Primitive data types are predefined by the language and named by a keyword. Let us now look into detail about the eight primitive data types.

➤ byte:

- Byte data type is an 8-bit signed two's complement integer.

- Minimum value is -128 (-2^7)

- Maximum value is 127 (inclusive) ($2^7 - 1$)

- Default value is 0

- Byte data type is used to save space in large arrays, mainly in place of integers, since a byte is four times smaller than an int.

Example: byte a = 100, byte b = -50

➤ short:

- Short data type is a 16-bit signed two's complement integer.

- Minimum value is -32,768 (-2^{15})

- Maximum value is 32,767 (inclusive) ($2^{15} - 1$)

Short data type can also be used to save memory as byte data type. A short is 2 times smaller than an int

- Default value is 0.

Example: short s = 10000, short r = -20000

➤ int:

- int data type is a 32-bit signed two's complement integer.

- Minimum value is -2,147,483,648 (-2^{31})

- Maximum value is 2,147,483,647 (inclusive) ($2^{31} - 1$)

- int is generally used as the default data type for integral values unless there is a concern about memory.

- The default value is 0.

Example: int a = 100000, int b = -200000

➤ long:

- Long data type is a 64-bit signed two's complement integer.
- Minimum value is -9,223,372,036,854,775,808. (-2^{63})
- Maximum value is 9,223,372,036,854,775,807 (inclusive). ($2^{63} - 1$)
- This type is used when a wider range than int is needed.
- Default value is 0L.

Example: int a = 100000L, int b = -200000L

➤ float:

- Float data type is a single-precision 32-bit IEEE 754 floating point.
- Float is mainly used to save memory in large arrays of floating point numbers. - Default value is 0.0f.
- Float data type is never used for precise values such as currency.

Example: float f1 = 234.5f

➤ double:

- double data type is a double-precision 64-bit IEEE 754 floating point.
- This data type is generally used as the default data type for decimal values , generally the default choice.
- Double data type should never be used for precise values such as currency.
- Default value is 0.0d.

Example: double d1 = 123.4

➤ boolean:

- boolean data type represents one bit of information.
- There are only two possible values: true and false.
- This data type is used for simple flags that track true/false conditions. Default value is false.

Example: boolean one = true

➤ char:

- char data type is a single 16-bit Unicode character.

- Minimum value is '\u0000' (or 0).
- Maximum value is '\uffff' (or 65,535 inclusive).
- Char data type is used to store any character.

Example: char letterA ='A'

Reference Data Types:

- ✓ Reference variables are created using defined constructors of the classes. They are used to access objects. These variables are declared to be of a specific type that cannot be changed. For example, Employee, Puppy , etc.

Class objects and various types of array variables come under reference data type.

Default value of any reference variable is null.

A reference variable can be used to refer to any object of the declared type or any compatible type.

Example: Animal animal = new Animal("giraffe");

The concepts of data types, variables, and constants:

Data Types

- A data type is a classification of data that determines the type of value a variable can hold and the operations that can be performed on it. Common data types include:
 - Primitive types (e.g., int, double, boolean)
 - Reference types (e.g., String, arrays, objects)

Variables

- ✓ A variable is a named storage location that holds a value of a specific data type. Variables have:

- A name (identifier)

- A data type

- A value (which can change)

Variables are used to store and manipulate data in a program.

Constants

A constant is a named value that cannot be changed once it is defined. Constants have:

- A name (identifier)

- A value (which is fixed and unchangeable)

- A data type (implicit or explicit)

Constants are used to represent fixed values that don't change during program execution.

Key differences:

- Variables can change value, constants can't.
- Variables have a changing state, constants have a fixed state.

Examples:

- Variable: `int age = 25;` (value can change)
- Constant: `final int MAX_WIDTH = 800;` (value is fixed)

In summary:

- Data types define the type of data
- Variables store and manipulate data
- Constants represent fixed, unchangeable values

Field and Constructor in Java

In Java, fields and constructors are essential components of classes:

Fields:

- Also known as data members or attributes
- Variables declared inside a class, outside any method or block
- Represent the properties or state of an object
- Can be:
 - Primitive types (int, double, boolean, etc.)
 - Reference types (objects, arrays, etc.)
- Can have access modifiers (public, private, protected, default)
- Can be initialized with default values or in constructors

Constructors:

- Special methods that initialize objects when created
- Have the same name as the class
- No return type, not even void
- Called when an object is instantiated (created)
- Can have parameters (overloaded constructors)
- Can initialize fields, perform calculations, or call other methods

Key points:

- Fields store data, while constructors initialize that data
- Constructors are called when an object is created, ensuring proper initialization

- Overloaded constructors allow for flexibility in object creation

Example:

```

...
public class Car {
    private String color; // field
    private int year; // field

    // Constructor

    public Car(String color, int year) {
        this.color = color;
        this.year = year;
    }
}
...

```

In this example, 'color' and 'year' are fields, and the 'Car' constructor initializes these fields when a new 'Car' object is created.

Concepts of methods

Methods are blocks of code that perform a specific task or set of tasks. Here are some key concepts related to methods:

1. ***Method Declaration*:** A method is declared with a name, return type, and parameters.

Here is an example of a method declaration in Java:

```

...
public int add(int a, int b) {
    return a + b;
}
...

```

In this example:

- 'public' is the access modifier, indicating that the method can be accessed from outside the class.
- 'int' is the return type, indicating that the method returns an integer value.
- 'add' is the method name.

- `(int a, int b)` is the parameter list, indicating that the method takes two integer parameters, `a` and `b`.

- `return a + b;` is the method body, which contains the code to be executed when the method is called.

This method can be called by passing two integer arguments, like this:

...

```
int result = add(2, 3);
```

```
System.out.println(result); // Output: 5
```

...

Note that the method declaration includes the following elements:

- Access modifier (public)
- Return type (int)
- Method name (add)
- Parameter list ((int a, int b))
- Method body (return a + b;)

2. ***Method Signature***: The method signature consists of the method name and parameter list.

3. ***Return Type***: The return type specifies the type of value a method returns.

4. ***Parameters***: Parameters are the input values passed to a method.

5. ***Method Body***: The method body contains the code that is executed when the method is called.

6. ***Method Call***: A method is called by its name, followed by parentheses containing the arguments.

7. ***Arguments***: Arguments are the actual values passed to a method when it's called.

8. ***Method Overloading***: Multiple methods with the same name but different parameter lists.

9. ***Method Overriding***: A subclass provides a different implementation of a method already defined in its superclass.

10. ***Static Methods***: Belong to the class, not instances, and can be called without creating an object.

11. ***Instance Methods***: Belong to instances of a class and require an object to be called.

12. ***Abstract Methods***: Declared without an implementation and must be implemented by subclasses.

13. ***Final Methods***: Cannot be overridden by subclasses.

The concept of overloading methods, garbage collection and nested classes

Here's an explanation of each concept:

Method Overloading

Method overloading is a feature in Java that allows multiple methods with the same name to be defined, as long as they have different parameter lists. This means that a class can have multiple methods with the same name, but each method must have a unique signature (name and parameter list).

Example:

...

```
public class Calculator {  
    public int add(int a, int b) {  
        return a + b;  
    }  
    public double add(double a, double b) {  
        return a + b;  
    }  
    public int add(int a, int b, int c) {  
        return a + b + c;  
    }  
}
```



```
}  
...  
...
```

In this example, the ``Calculator`` class has three ``add`` methods with different parameter lists.

Garbage Collection

✓ Garbage collection is the process by which Java automatically reclaims memory occupied by objects that are no longer needed. This helps to prevent memory leaks and reduce the risk of running out of memory.

Here's how it works:

1. Java keeps track of all objects created in memory.
2. When an object is no longer referenced or needed, it becomes eligible for garbage collection.
3. The garbage collector periodically runs in the background to identify and remove eligible objects.
4. The memory occupied by removed objects is then made available for new objects.

Nested Classes

Nested classes are classes defined inside another class. They are also known as inner classes. There are two types of nested classes:

1. ***Static Nested Classes***: These are nested classes that are defined with the ``static`` keyword. They can be accessed without creating an instance of the outer class.
2. ***Non-Static Nested Classes*** (Inner Classes): These are nested classes that are not defined with the ``static`` keyword. They require an instance of the outer class to be accessed.

Example:

...

```
public class OuterClass {
```

```
    public static class StaticNestedClass {
```

```
        // ...
```

```
    }
```

```
public class InnerClass {
```

```
    // ...
```

```
}
```

```
}
```

```
...
```

In this example, `StaticNestedClass` is a static nested class, and `InnerClass` is a non-static nested class.

local and instance variables

Here's a differentiation between local and instance variables:

Local Variables

- Declared within a method or block
- Only accessible within that method or block
- Created when the method or block is executed
- Destroyed when the method or block completes execution
- Do not retain their values between method calls
- Are not associated with any object or instance

Instance Variables

- Declared within a class, but outside any method or block
- Associated with an instance of the class (an object)
- Each object has its own copy of the instance variables
- Retain their values between method calls
- Are accessible from any method within the class
- Are created when an object is created and destroyed when the object is garbage collected

Key differences:

- Scope: Local variables have a limited scope, while instance variables have a broader scope.
- Lifetime: Local variables are short-lived, while instance variables live as long as the object exists.
- Accessibility: Local variables are only accessible within the method or block, while instance variables are accessible from any method within the class.

Example:

...

```
public class MyClass {
    // Instance variable
    private int instanceVar;

    public void myMethod() {
        // Local variable
        int localVar = 10;
        instanceVar = 20;
    }
}
```

'''
In this example, 'instanceVar' is an instance variable, and 'localVar' is a local variable.

Distinguish between private and public modifiers.

In Java, modifiers are keywords that alter or modify the behavior of a class, method, or variable. Here are the types of modifiers in Java:

1. Access Modifiers:

- public: Accessible from anywhere
- private: Accessible only within the same class
- protected: Accessible within the same class and subclasses
- default (no modifier): Accessible within the same package

2. Non-Access Modifiers:

- static: Belongs to the class, not instances
- final: Cannot be overridden or changed
- abstract: Cannot be instantiated, must be subclassed
- synchronized: Thread-safe, only one thread can access at a time
- volatile: Indicates that a variable may change unexpectedly
- transient: Excluded from serialization

3. Other Modifiers:

- strictfp: Restricts floating-point calculations to ensure portability
- native: Implemented in native code, not Java

Modifiers can be combined in various ways to achieve the desired behavior. For example:

'''

```
public static final int MAX_VALUE = 100;
'''
```

In this example, the 'MAX_VALUE' variable is declared with the 'public', 'static', and 'final' modifiers, making it a constant that can be accessed from anywhere.

Here's a distinction between private and public modifiers:

_Private Modifier

- Accessible only within the same class
- Cannot be accessed directly from outside the class
- Can be accessed through public methods (getters and setters) within the class
- Provides encapsulation and data hiding
- Typically used for instance variables and helper methods

_Public Modifier

- Accessible from anywhere in the program
- Can be accessed directly from outside the class
- Provides no encapsulation or data hiding
- Typically used for methods that need to be accessed from outside the class

Key differences:

- **Accessibility:** Private members are only accessible within the class, while public members are accessible from anywhere.
- **Encapsulation:** Private members provide encapsulation, while public members do not.

Example:

```
'''
```

```
public class MyClass {  
  
    private int privateVar; // Only accessible within MyClass  
    public int publicVar; // Accessible from anywhere
```

```
    private void privateMethod() { // Only callable within MyClass  
        // ...  
    }  
}
```

```
    public void publicMethod() { // Callable from anywhere  
        // ...  
    }  
}
```

```
...  
}
```

In this example, `privateVar` and `privateMethod()` are only accessible within `MyClass`, while `publicVar` and `publicMethod()` are accessible from anywhere in the program.

Describe parameter passing method

In Java, there are two parameter passing methods:

1. *Pass by Value* (Primitive Types):

- A copy of the original value is passed to the method.
- Changes made to the parameter within the method do not affect the original value.
- Only primitive types (int, double, boolean, etc.) are passed by value.

2. *Pass by Reference* (Object Types):

- A copy of the original reference is passed to the method.
- Changes made to the object's state within the method affect the original

object.

- Object types (arrays, classes, interfaces, etc.) are passed by reference.

Note that Java does not support "pass by reference" for primitive types, only for object types. However, the reference itself is passed by value.

Here's an example to illustrate the difference:

```
...  
  
public class ParameterPassing {  
    public static void main(String[] args) {  
        int x = 10; // primitive type  
        changePrimitive(x);  
  
        System.out.println(x); // prints 10  
  
        Person person = new Person("John"); // object type  
        changeObject(person);  
  
        System.out.println(person.name); // prints "Jane"  
    }  
  
    public static void changePrimitive(int x) {  
        x = 20; // changes the local copy, not the original  
    }  
  
    public static void changeObject(Person person) {  
        person.name = "Jane"; // changes the original object's state  
    }  
}
```

```
}
```

```
class Person {
```

```
    String name;
```

```
    public Person(String name) {
```

```
        this.name = name;
```

```
    }
```

```
}
```

```
...
```

In this example, the 'changePrimitive' method does not affect the original value of 'x', while the 'changeObject' method changes the state of the original 'person' object.

Operator in Java

An operator is a symbol that tells the compiler to perform specific mathematical or logical manipulations. Java is rich in built-in operators and provide the following types of operators –

Arithmetic Operators

There are following arithmetic operators supported by C++ language –

Assume variable A holds 10 and variable B holds 20, then –

Examples

Operator	Description	Example
+	Adds two operands	A + B will give 30

-	Subtracts second operand from the first	A - B will give -10
*	Multiplies both operands	A * B will give 200
/	Divides numerator by denominator	B / A will give 2
%	Modulus Operator and remainder of after an integer division	B % A will give 0
++	Increment operator, increases integer value by one	A++ will give 11
--	Decrement operator, decreases integer value by one	A-- will give 9

Relational Operators

Assume variable A holds 10 and variable B holds 20, then –

Show Examples

Operator	Description	Example
==	Checks if the values of two operands are equal or not, if yes then condition becomes true.	(A == B) is not true.
!=	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.	(A != B) is true.
>	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	(A > B) is not true.
<	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	(A < B) is true.
>=	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	(A >= B) is not true.
<=	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	

Logical Operators

There are following logical operators supported by C++ language.

Assume variable A holds 1 and variable B holds 0, then –

Show Examples

Operator	Description	Example
&&	Called Logical AND operator. If both the operands are non-zero, then condition becomes true.	(A && B) is false.
	Called Logical OR Operator. If any of the two operands is non-zero, then condition becomes true.	(A B) is true.
!	Called Logical NOT Operator. Use to reverses the logical state of its operand. If a condition is true, then Logical NOT operator will make false.	!(A && B) is true.

Assignment Operators

There are following assignment operators supported by C++ language –

Show Examples

Operator	Description	Example
=	Simple assignment operator, Assigns values from right side operands to left side operand.	C = A + B will assign value of A + B into C
+=	Add AND assignment operator, It adds right operand to the left operand and assign the result to left operand.	C += A is equivalent to C = C + A
-=	Subtract AND assignment operator, It subtracts right operand from the left operand and assign the result to left operand.	C -= A is equivalent to C = C - A
*=	Multiply AND assignment operator, It multiplies right operand with the left operand and assign the result to left operand.	C *= A is equivalent to C = C * A
/=	Divide AND assignment operator, It divides left operand with the right operand and assign the result to left operand.	C /= A is equivalent to C = C / A
%=	Modulus AND assignment operator, It takes modulus using two operands and assign the result to left operand.	C %= A is equivalent to C = C % A
<<=	Left shift AND assignment operator.	C <<= 2 is same as C = C << 2
>>=	Right shift AND assignment operator.	C >>= 2 is same as C = C >> 2

<code>&=</code>	Bitwise AND assignment operator.	<code>C &= 2</code> is same as <code>C = C & 2</code>
<code>^=</code>	Bitwise exclusive OR assignment operator.	<code>C ^= 2</code> is same as <code>C = C ^ 2</code>
<code> =</code>	Bitwise inclusive OR assignment operator.	<code>C = 2</code> is same as <code>C = C 2</code>

Expressions in java using precedence rules.

Expressions in programming languages follow a set of rules called precedence rules, which determine the order in which operators are evaluated. Here's a breakdown of the precedence rules:

1. ***Parentheses***: Evaluate expressions inside parentheses first.
2. ***Unary operators***: Evaluate unary operators (e.g., ``-``, ``!``, ``++``, ``--``) next.
3. ***Multiplicative operators***: Evaluate multiplicative operators (e.g., ``*``, ``/``, ``%``) from left to right.
4. ***Additive operators***: Evaluate additive operators (e.g., ``+``, ``-``) from left to right.
5. ***Relational operators***: Evaluate relational operators (e.g., ``==``, ``!=``, ``>``, ``<``) from left to right.
6. ***Logical operators***: Evaluate logical operators (e.g., ``&&``, ``||``) from left to right.
7. ***Assignment operators***: Evaluate assignment operators (e.g., ``=``, ``+=``, ``-=``, ``*=``, ``/=``) last.

Here's an example expression: ``3 + 4 * 2 - 1``

Using the precedence rules, we evaluate it as follows:

1. Parentheses: None
2. Unary operators: None
3. Multiplicative operators: ``4 * 2 = 8``
4. Additive operators: ``3 + 8 = 11``

5. Subtract: `11 - 1 = 10`

Final result: `10`

By following the precedence rules, we ensure that expressions are evaluated Java using precedence rules.

Describe Boolean expressions using relational and logical operators.

Boolean expressions using relational and logical operators are used to evaluate conditions and make decisions in programming. Here's a breakdown:

Relational Operators:

1. `==` (equal to)
2. `!=` (not equal to)
3. `>` (greater than)
4. `<` (less than)
5. `>=` (greater than or equal to)
6. `<=` (less than or equal to)

These operators compare two values or expressions, returning `true` or `false`.

Logical Operators:

1. `&&` (logical AND)
2. `||` (logical OR)
3. `!` (logical NOT)

These operators combine Boolean values or expressions, returning `true` or `false`.

Boolean Expressions:

Combine relational and logical operators to create Boolean expressions:

1. `x > 5 && y < 10` (true if x is greater than 5 and y is less than 10)
2. `a == 5 || b != 3` (true if a is equal to 5 or b is not equal to 3)

3. `!(x > 5)` (true if x is not greater than 5)

Evaluation Rules:

1. Evaluate relational operators first (from left to right)
2. Evaluate logical operators next (from left to right)
3. Use parentheses to override the default order of operations

Examples:

1. `x > 5 && y < 10 || z == 2` (true if (x > 5 and y < 10) or z is equal to 2)
2. `!(x > 5 && y < 10)` (true if not (x > 5 and y < 10))

By combining relational and logical operators, you can create complex Boolean expressions to control the flow of your program.

Conditional statement in Java

In Java, conditional statements are used to execute different blocks of code based on specific conditions or decisions. There are several types of conditional statements in Java:

1. ***If Statement***: Executes a block of code if a condition is true.

```
if (condition) {
```

```
    // code to execute
```

```
}
```

1. ***If-Else Statement***: Executes one block of code if a condition is true and another block if it's false.

```
if (condition) {
```

```
    // code to execute if true
```

```
} else {
```

```
    // code to execute if false
```

```
}
```

1. ***If-Else-If Statement***: Executes different blocks of code based on multiple conditions.

```
""
if (condition1) {
    // code to execute if condition1 is true
} else if (condition2) {
    // code to execute if condition1 is false and condition2 is true
} else {
    // code to execute if all conditions are false
}
""
```

1. ***Switch Statement***: Executes different blocks of code based on the value of a variable.

```
""
switch (variable) {
    case value1:
        // code to execute if variable equals value1
        break;
    case value2:
        // code to execute if variable equals value2
        break;
    default:
        // code to execute if variable doesn't match any case
}
""
```

1. ***Ternary Operator***: A concise way to execute a simple if-else statement.

```
""  
""  
condition ? code_to_execute_if_true : code_to_execute_if_false;  
""
```

These conditional statements allow you to control the flow of your program's execution based on different conditions, making your code more dynamic and flexible.

Concept of looping in Java

Looping is a fundamental concept in programming that allows you to execute a block of code repeatedly for a specified number of times or until a certain condition is met. It enables you to automate repetitive tasks, process large datasets, and create dynamic effects.

Types of Loops:

1. ***For Loop***: Executes a block of code for a specified number of iterations, typically using a counter variable.

Here is an example of a for loop in Java:

```
""
```

Example

```
for (int i = 0; i < 10; i++) {
```

```
    System.out.println("Iteration " + i);
```

```
}
```

```
""
```

This loop will output:

```
""
```

Iteration 0

Iteration 1

Iteration 2

Iteration 3

Iteration 4

```
Iteration 5
Iteration 6
Iteration 7
Iteration 8
Iteration 9
'''
```

Explanation:

- ``int i = 0``: Initializes the loop counter ``i`` to 0.
- ``i < 10``: Sets the condition for the loop to continue as long as ``i`` is less than 10.
- ``i++``: Increments the loop counter ``i`` by 1 after each iteration.

The loop body ``System.out.println("Iteration " + i);`` is executed 10 times, with ``i`` taking on the values 0 through 9.

Here's another example that uses a for loop to iterate over an array:

```
'''
```

```
String[] colors = {"red", "green", "blue", "yellow"};
for (int i = 0; i < colors.length; i++) {
    System.out.println(colors[i]);
}
'''
```

This loop will output:

```
'''
```

```
red
green
blue
yellow
'''
```

In this example, the loop iterates over the ``colors`` array, printing each element in

turn. The loop counter `i` takes on the values 0 through 3, corresponding to the indices of the array elements.

2. ***While Loop***: Executes a block of code as long as a specified condition is true.

Here is an example of a while loop in Java:

```
""  
int i = 0;  
while (i < 10) {  
    System.out.println("Iteration " + i);  
    i++;  
}  
""
```

This loop will output:

```
""  
Iteration 0  
Iteration 1  
Iteration 2  
Iteration 3  
Iteration 4  
Iteration 5  
Iteration 6  
Iteration 7  
Iteration 8  
Iteration 9  
""
```

Explanation:

- `int i = 0`: Initializes the loop counter `i` to 0.
- `while (i < 10)`: Sets the condition for the loop to continue as long as `i` is less than 10.
- `System.out.println("Iteration " + i)`: Prints the current iteration number.
- `i++`: Increments the loop counter `i` by 1 after each iteration.

The loop continues to execute as long as the condition `i < 10` is true. Once `i` reaches 10, the loop terminates.

3. ***Do-While Loop***: Executes a block of code at least once and then repeats it while a specified condition is true.

Example Do-While Loop:

Here is an example of a do-while loop in Java:

```
int i = 0;

do {
```

```
    System.out.println("Iteration " + i);
```

```
    i++;
```

```
} while (i < 10);
```

```
````
```

This loop will output:

```
````
```

Iteration 0

Iteration 1

Iteration 2

Iteration 3

Iteration 4

Iteration 5

Iteration 6

Iteration 7

Iteration 8

Iteration 9

...

Explanation:

- ``int i = 0``: Initializes the loop counter ``i`` to 0.
- ``do { ... }``: Executes the loop body at least once before checking the condition.
- ``System.out.println("Iteration " + i);``: Prints the current iteration number.
- ``i++``: Increments the loop counter ``i`` by 1 after each iteration.
- ``while (i < 10)``: Sets the condition for the loop to continue as long as ``i`` is less than 10.

The loop body is executed at least once, and then the condition is checked. If the condition is true, the loop body is executed again. This process repeats until the condition becomes false.

Note that the main difference between a do-while loop and a while loop is that a do-while loop executes the loop body at least once before checking the condition, whereas a while loop checks the condition before executing the loop body.

Here's another example that uses a do-while loop to simulate a game:

...

```
int playerHealth = 100;
```

```
do {
```

```
    System.out.println("Player health: " + playerHealth);
```

```
    playerHealth -= 10; // simulate damage
```

```
} while (playerHealth > 0);
```

```
System.out.println("Game over!");
```

...

This loop continues to simulate damage to the player's health until it reaches 0, at which point the game ends.

Loop Components:

1. ***Initialization***: Sets the starting value of the loop counter or condition.
2. ***Condition***: Evaluates whether the loop should continue or terminate.
3. ***Increment/Decrement***: Updates the loop counter or condition for the next iteration.
4. ***Loop Body***: The code executed on each iteration.

Looping Benefits:

1. ***Automation***: Repeats tasks without manual intervention.
2. ***Efficiency***: Reduces code duplication and improves performance.
3. ***Flexibility***: Allows for dynamic changes to the loop behavior.

Common Looping Scenarios:

1. ***Array/Collection Processing***: Iterates through elements to perform operations.
2. ***File Processing***: Reads or writes files line-by-line or chunk-by-chunk.
3. ***Game Development***: Updates game state, animates graphics, or handles user input.
4. ***Data Analysis***: Processes large datasets, performs calculations, or generates reports.

By leveraging looping constructs, you can write more efficient, effective, and scalable code to tackle a wide range of programming challenges.